PROJE 10 / 20 · ÇALIŞAN TEMEL SÜRÜM

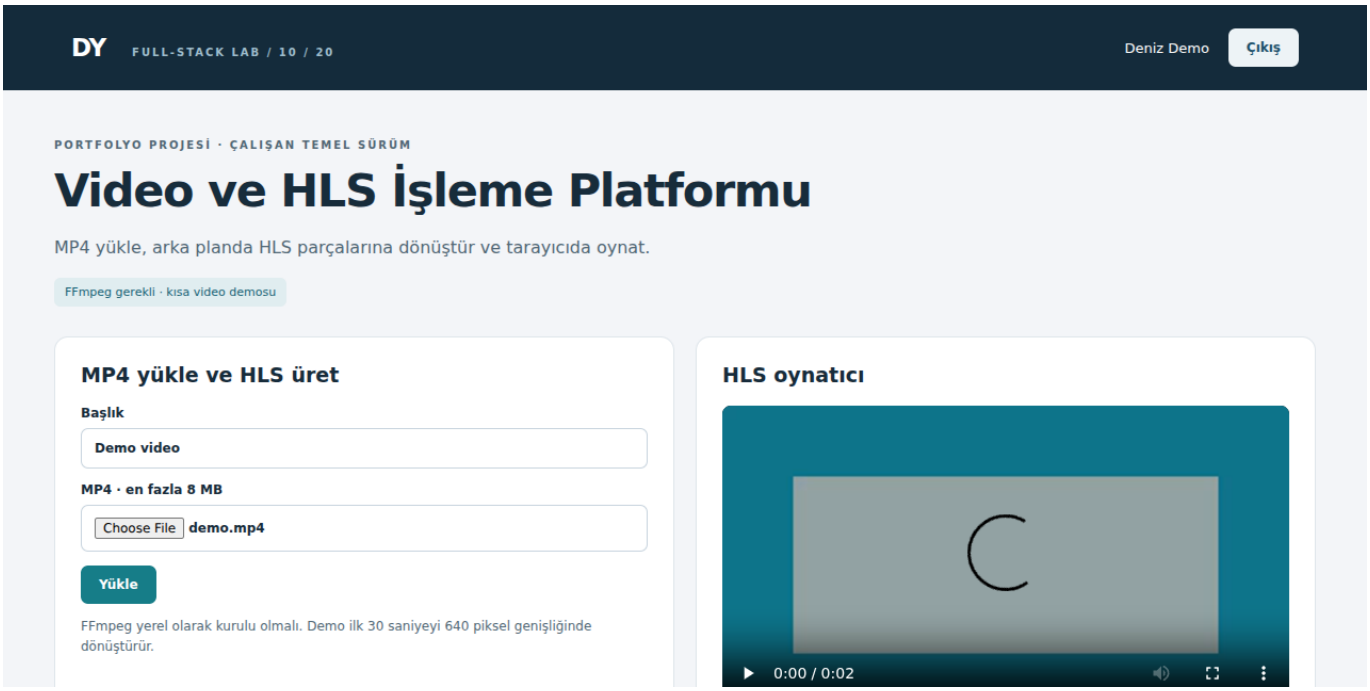
# Video ve HLS İşleme Platformu

MP4 yükle, arka planda HLS parçalarına dönüştür ve tarayıcıda oynat.

FFmpeg gerekli · kısa video demosu

## Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.




Gerçek uygulama ekranı · yerel demo verileri

## Teslim edilen kapsam

8 MB MP4 yükleme · Kalıcı işlem durumu · Tek worker kuyruğu · FFmpeg ile HLS · hls.js oynatıcı

## Bu sürümde olmayanlar

FFmpeg makinede kurulu olmalı. İlk 30 saniye, 640 piksel, tek kalite profili. Dockerfile FFmpeg içermez. Gerçek streaming CDN, adaptif kalite merdiveni ve BullMQ yok. Dosya HTTP JSON base64 ile yüklenir.

# 02 / Backend ve veri modeli

ftyp MP4 başlığı ve boyut kontrol edilir. FFmpeg shell kullanılmadan argüman dizisiyle başlatılır. Dosya protokolleri sınırlandırılır, işlem 60 saniyede kesilir. Kullanıcı yalnız kendi video parçalarını okuyabilir; parça adları izinli desenle eşleşmelidir. Yeniden başlatmada yarım kalan işler hata olarak işaretlenir.

## Gerçek SQL şeması

```
CREATE TABLE videos(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),title TEXT,status
 TEXT,error TEXT,created_at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

## Kod okuma sırası

| Dosya | Ne arayacaksın? |
| --- | --- |
| client/Workspace.vue | Eylemi tetikleyen form ve API isteği |
| client/api.ts | JSON isteği, hata yakalama ve mesaj |
| server/project.js | Yetki kontrolü → kural → SQL → yanıt |
| server/core.js | Parametreli SQL, transaction, oturum, SSE |

## Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

# 03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

| Yöntem / yol | Girdi alanları | Başarı |
|---|---|---|
| GET /api/videos | Gövde yok | 200 |
| POST /api/videos | JSON: title, base64 | 202 |
| GET /api/videos/:id/file/:name | Gövde yok | 200 |

## Somut istek örneği

```
POST /api/videos
{"title":"Demo","base64":"<mp4-base64>"}
202 {"id":"<video-id>"}
```

## Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

| Kod | Anlam |
|---|---|
| 401 / 403 | Giriş gerekli / rol veya origin reddedildi |
| 404 / 409 | Kayıt görünmüyor / durum veya benzersizlik çakışması |
| 422 / 429 | Geçersiz girdi / giriş veya uygulama kotası |
| 500 / 502 / 503 | Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis |

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

# 04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- MP4 olmayan dosya 422 ile reddedilmeli.
- FFmpeg varsa örnek MP4 HLS ready durumuna gelmeli.
- HLS manifesti ve parçalar yalnız sahibine sunulmalı.
- Video HLS oynatıcıda yüklenebilmeli.

## Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

## Test kodundan örnek

```
await post('/videos',{title:'Bad',base64:Buffer.from('not an mp4
video').toString('base64')},422);
assert.equal((await get('/videos')).length,0);
await call('/api/videos/unknown/file/input.mp4','GET',undefined,404);
```

## Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

## Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

# 05 / Çalışma ve GitHub planı

## İlk günün somut hedefi

fixtures/demo.mp4 dosyasını yükle. queued/processing/ready akışını izle; HLS manifest ve .ts isteklerini tarayıcıda incele.

| Çalışma bloğu | Teslim edeceğin çıktı |
| --- | --- |
| 1 · Kurulum / 30-60 dk | Çalışan ekran, örnek veri ve ilk başarılı API isteği |
| 2 · Kod okuma / 45-90 dk | Formdan SQL sorgusuna kadar bir isteğin açıklaması |
| 3 · Geliştirme / 60-120 dk | Bir iş kuralı veya arayüz davranışında kendi değişikliğin |
| 4 · Doğrulama / 30-60 dk | İlgili test ve README değişikliği; yeni ekran görüntüsü |

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

## İleri sürüm: dört gerçek geliştirme

- 1. Multipart upload ekle
- 2. Kalıcı kuyruk ve ayrı worker kullan
- 3. Çoklu kalite profili üret
- 4. Obje deposu, süreli URL ve CDN ekle

## Anlamlı commit sırası

```
git add .
git commit -m "chore: add video-hls baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

## Mülakatta anlat

Video dönüşümü neden HTTP isteğini bekletmemeli? Adaptive bitrate için kaç manifest türü gerekir?

## Özel kurulum / ilerleme notu

FFmpeg komutunun PATH içinde bulunması gerekir: `ffmpeg -version`. Alternatif olarak FFMPEG_PATH ortam değişkenine tam çalıştırılabilir dosya yolunu ver. fixtures/demo.mp4 iki saniyelik sentetik videodur. Bu sürüm için Dockerfile kullanacaksan FFmpeg kurulumu eklemelisin.